

SMF Tools (SmfPlayer / MidiRecorder / TempoMapExtractor)

This page describes the components and utilities for playing, recording, and analyzing the tempo of Standard MIDI Files (SMF).

Namespaces:

- Components: `jp.kshoji.unity.midi`
- Utilities: `jp.kshoji.unity.midi.util`

Location:

- `Assets/MIDI/Scripts/Gameplay/SmfPlayer.cs`
- `Assets/MIDI/Scripts/Gameplay/MidiRecorder.cs`
- `Assets/MIDI/Scripts/MidiSequenceAsset.cs`
- `Assets/MIDI/Scripts/UmpSequenceAsset.cs`
- `Assets/MIDI/Scripts/Editor/UmpImportUtility.cs` / `UmpAssetImportMenu.cs`
- `Assets/MIDI/Scripts/Utilities/Smf/TempoMapExtractor.cs`, etc.

Overview

Component / Utility	Role
MidiSequenceAsset	Hold SMF binary data as a ScriptableObject
UmpSequenceAsset	Hold <code>.midi2</code> (UMP MIDI Clip) binary data as a ScriptableObject
SmfPlayer	Play SMF / Sequence with <code>SequencerImpl</code> and output to <code>MidiManager</code>
MidiRecorder	Record live MIDI input as SMF
TempoMapExtractor	Extract tempo and time signatures from a Sequence as a time series

All of these wrap the existing `jp.kshoji.midisystem` (`Sequence` / `SequencerImpl`). For details on the low-level SMF I/O API, see [SMF / Sequencing](#).

MidiSequenceAsset

A ScriptableObject that holds SMF data as a Unity asset. Use it as a playback source for `SmfPlayer` or as a save destination for `MidiRecorder`.

How to create

- Inspector: `Assets > Create > MIDI > Sequence Asset`
- Code: `MidiSequenceAsset.CreateFromSequence(sequence, sourcePath)`

API

Method	Description
<code>SetSmfData(byte[] data, string sourcePath)</code>	Set the SMF binary data
<code>ToSequence()</code>	Convert to a <code>Sequence</code>
<code>CreateFromSequence(Sequence, string)</code>	Create an asset from a <code>Sequence</code>

UmpSequenceAsset

A ScriptableObject that holds `.midi2` (UMP MIDI Clip / SMF2CLIP) binary data as a Unity asset. Use it as a playback source for `UmpSequencer` or the Scriptable Audio `MidiDspUmpSequenceScheduler`.

How to create

- Inspector: `Assets > Create > MIDI > UMP Sequence Asset` (empty asset)
- From a file: `Assets > Create > MIDI > Import UMP Sequence Asset From File` (select a `.midi2`)
- Code: `UmpSequenceAsset.CreateFromSequence(umpSequence, sourcePath) / UmpImportUtility`

API

Method / Property	Description
<code>SetMidi2Data(byte[] data, string sourcePath, int index)</code>	Set the <code>.midi2</code> binary data
<code>ToUmpSequence()</code>	Convert the configured clip to a <code>UmpSequence</code>
<code>ToUmpSequenceList()</code>	Convert all clips in the file
<code>CreateFromSequence(UmpSequence, string)</code>	Create an asset from a <code>UmpSequence</code>
<code>ClipIndex</code>	Index within a multi-clip file (usually 0)

Usage example with Scriptable Audio

```
var asset = /* UmpSequenceAsset assigned in the Inspector */;
var bootstrap = GetComponent<MidiDspUmpSequenceBootstrap>();
bootstrap.sequence = asset.ToUmpSequence();
bootstrap.EnsureReady();
bootstrap.Scheduler.Play();
```

For details on DSP-synchronized playback, see [Unity Ecosystem Integration — UMP sequence](#). For wall-clock playback, see [MIDI 2.0 — UmpSequencer](#).

SmfPlayer

A component that integrates `SequencerImpl` into the Unity lifecycle and plays SMF synchronized to game time. Output is sent to `MidiManager` via the internal `MidiManagerReceiverBridge`.

Setup

1. Attach `SmfPlayer` to a `GameObject`
2. Assign a `MidiSequenceAsset` to `Sequence Asset` (or call `SetSequence()` at runtime)
3. Specify `Output Device Id` (empty = first output device)
4. Call `MidiManager.InitializeMidi()` in your scene bootstrap

MPTK audio output (`FEATURE_USE_MPTK`)

To output to an MPTK synth instead of a hardware MIDI device:

1. Enable `FEATURE_USE_MPTK` in Project Settings
2. Add `MptkSmfPlayerOutput` to the same `GameObject`
3. In Play mode, `Configure()` sets `outputDeviceId` to `mptk:internal`

For details, see [Maestro / MPTK Integration — SmfPlayer linkage](#).

Editor SMF preview

After loading a `.mid` from `Window > MIDI > SMF Preview`, you can preview it in Play Mode from **Preview Audio (MPTK)** (`FEATURE_USE_MPTK` required). For details, see [Maestro / MPTK Integration — Editor SMF preview](#).

Main settings

Field	Description
<code>sequenceAsset</code>	The SMF asset to play
<code>group</code>	MIDI group (0–15)
<code>outputDeviceId</code>	Output device ID (empty = auto-select)
<code>outputChannel</code>	Map all messages to this channel (<code>-1</code> = keep original channel)
<code>playOnAwake</code>	Auto-play after Awake
<code>loop</code> / <code>loopCount</code>	Loop playback (<code>-1</code> = infinite)
<code>tempoBpm</code>	Playback tempo (changes during runtime are also reflected)

Control API

Method	Description
<code>Play()</code>	Start playback (resumes from position when paused)
<code>Pause()</code>	Pause
<code>Stop()</code>	Stop and return to the beginning
<code>Seek(float timeSeconds)</code>	Seek the playback position
<code>SetTrackMute(index, mute)</code>	Mute a track
<code>SetTrackSolo(index, solo)</code>	Solo a track
<code>SetSequenceAsset(asset)</code>	Swap the asset and reload
<code>SetSequence(sequence)</code>	Directly specify a runtime Sequence

State properties

Property	Description
<code>State</code>	<code>Stopped</code> / <code>Playing</code> / <code>Paused</code>
<code>CurrentTimeSeconds</code>	Current playback position (seconds)
<code>TotalDurationSeconds</code>	Total playback time (seconds, based on <code>TempoMapExtractor</code>)

Events

UnityEvent	Timing
<code>onPlaybackStarted</code>	Playback started
<code>onPlaybackPaused</code>	Paused
<code>onPlaybackStopped</code>	Stopped
<code>onPlaybackFinished</code>	Reached the end without looping

Usage example

```
using jp.kshoji.unity.midi;
using UnityEngine;

public sealed class SmfPlaybackController : MonoBehaviour
{
    public SmfPlayer player;

    void Start()
    {
        MidiManager.Instance.InitializeMidi(() => player.Play());
    }

    public void OnSeek(float normalized)
    {
        player.Seek(player.TotalDurationSeconds * normalized);
    }
}
```

MidiRecorder

Records live MIDI input (Note / CC / Program Change / Pitch Bend / Aftertouch / SysEx / system realtime) to a `Sequence` and saves it as a `.mid` file or a `MidiSequenceAsset`.

The current implementation uses its own `Time.time`-based timestamping (integration with the `SequencerImpl` recording API is a future extension).

Setup

1. Attach `MidiRecorder` to a `GameObject`
2. Register with `MidiManager` via `registerWithMidiManager = true` (default)
3. Set `deviceIdFilter` / `channelFilter` as needed
4. Control recording with `StartRecording()` / `StopRecording()`

When using it via a filter chain, set `registerWithMidiManager = false` and place it under the upstream filter's `forwardTargets` (same as [Gameplay Components](#)).

Main settings

Field	Description
<code>tempoBpm</code>	Tempo at record time (used for tick calculation)
<code>ticksPerQuarter</code>	Resolution (PPQ, default 480)
<code>deviceIdFilter</code>	Devices to record (empty = all devices)
<code>channelFilter</code>	Channels to record (empty = all channels)

API

Method	Description
<code>StartRecording()</code>	Start recording
<code>StopRecording()</code>	Stop recording
<code>ClearRecording()</code>	Clear the recorded content
<code>GetSequence()</code>	Get the recorded <code>Sequence</code>
<code>SaveToFile(path)</code>	Save to a <code>.mid</code> file
<code>SaveToAsset(assetPath)</code>	Create a <code>MidiSequenceAsset</code> (Editor only)

Recording format

- SMF Format 1 (tempo track + data track)
- Recorded: Note On/Off, Control Change, Program Change, Pitch Bend, Channel/Poly Aftertouch, SysEx / System Common, MIDI realtime (Clock / Start / Stop, etc.)
- Raw UMP is not written to SMF (handled separately via `onMessage`)

Usage example

```
using jp.kshoji.unity.midi;
using UnityEngine;

public sealed class RecordingExample : MonoBehaviour
{
    public MidiRecorder recorder;
    public SmfPlayer player;
    public MidiSequenceAsset recordedAsset;

    public void RecordAndPlay()
    {
        recorder.StartRecording();
        // ... play MIDI ...
        recorder.StopRecording();

#if UNITY_EDITOR
        recordedAsset = recorder.SaveToAsset("Assets/Recorded.mid.asset");
        player.SetSequenceAsset(recordedAsset);
        player.Play();
#endif
    }
}
```

TempoMapExtractor

A static utility that extracts tempo changes and time signatures as a time series from a `Sequence` or SMF data. It can be used for calculating `SmfPlayer.TotalDurationSeconds` or as a foundation for score generation.

API

```
using jp.kshoji.unity.midi.util;
using jp.kshoji.midisystem;

TempoMap map = TempoMapExtractor.Extract(sequence);
// or
TempoMap map = TempoMapExtractor.Extract(smfBytes);
TempoMap map = TempoMapExtractor.Extract("/path/to/file.mid");
```

TempoMap properties / methods

Name	Description
<code>TempoChanges</code>	List of tempo changes
<code>TimeSignatures</code>	List of time signatures
<code>TotalDurationSeconds</code>	Total playback time (seconds)
<code>TotalTicks</code>	Total number of ticks
<code>TicksPerQuarter</code>	Resolution
<code>GetBpmAt(tick)</code>	BPM at the given tick
<code>TickToSeconds(tick)</code>	Tick → seconds
<code>SecondsToTick(seconds)</code>	Seconds → tick

For SMF without tempo meta-events, 120 BPM (500000 μ s/qn) is treated as the default.

Usage example

```
var map = TempoMapExtractor.Extract(sequence);
Debug.Log($"Duration: {map.TotalDurationSeconds:F2}s, BPM at 0: {map.GetBpmAt(0)}");

foreach (var signature in map.TimeSignatures)
{
    Debug.Log($"{{signature.TimeSeconds:F2}}s -> {{signature.Numerator}}/{{signature.Denom
```

MeasureTimeUtility

Bar / beat tick math and measure ↔ time conversion on a `TempoMap`.

Location: `Assets/MIDI/Scripts/Utilities/Smf/MeasureTimeUtility.cs`

Method	Description
<code>TicksPerBar(ppqn, numerator, denominator)</code>	Length of one bar in ticks
<code>TicksForBars(...)</code>	Length of N bars
<code>TicksPerBeat(...)</code>	Beat length (bar / numerator) for simple meters
<code>MeasureToStartSeconds</code> / <code>MeasureToEndSeconds</code>	1-based measure → seconds
<code>TryResolveMeasureRange</code> / <code>TryGetMeasureRangeMs</code>	Inclusive measure ranges
<code>TimeSecondsToMeasure</code>	Seconds → 1-based measure

Formula for bar ticks: `ppqn * 4 * numerator / denominator` (integer division).

TupletUtility / SwingUtility

Location: `Assets/MIDI/Scripts/Utilities/Smf/TupletUtility.cs` , `SwingUtility.cs`

TupletUtility

Rational tuplets on a fixed PPQN grid (e.g. 3:2 eighth-note triplets). Step ticks use integer division so index→tick→index round-trips for in-range indices.

Method	Description
<code>UnitTicks</code> / <code>GroupDurationTicks</code> / <code>StepTicks</code>	Unit, group span, and per-step ticks
<code>TickAtIndex</code> / <code>IndexAtTick</code>	Bidirectional mapping
<code>QuantizeToGroup</code>	Snap a tick onto the nearest tuplet step
<code>IsIndexTickReversible</code>	Verify round-trip for all indices

SwingUtility

Playback-time swing: delays even offbeats without rewriting edit ticks. `swingAmount` ≈ 0.33 yields a triplet-like feel.

Method	Description
<code>UnitTicksFromDivision(ppqn, division)</code>	Unit length (4 = quarter, 8 = eighth, ...)
<code>ApplySwing(editTick, unitTicks, amount)</code>	Map edit tick → swung playback tick
<code>ApplySwing(..., ppqn, swingDivision, amount)</code>	Convenience overload

MidiMetaMessageFactory

Builds and reads common SMF `MetaMessage` payloads.

Location: `Assets/MIDI/Scripts/Utilities/Smf/MidiMetaMessageFactory.cs`

Method	Description
<code>CreateTempo(bpm)</code>	Set Tempo meta (µs per quarter)
<code>CreateTimeSignature(numerator, denominator, ...)</code>	Time Signature meta
<code>CreateTrackName(name)</code>	Track / Sequence Name (UTF-8)
<code>DenominatorToExponent</code> / <code>ExponentToDenominator</code>	SMF power-of-two conversions
<code>TryReadTempoBpm(meta, out bpm)</code>	Read BPM from a Tempo meta

Source files

File	Role
MidiSequenceAsset.cs	SMF ScriptableObject
UmpSequenceAsset.cs	UMP .midi2 ScriptableObject
Editor/UmpImportUtility.cs / UmpAssetImportMenu.cs	.midi2 import
SmfPlayer.cs / SmfPlayerState.cs	SMF playback component
MidiManagerReceiverBridge.cs	Sequencer → MidiManager bridge
MidiRecorder.cs	Recording component
MidiRecordingSession.cs	Recording session (tick calculation / SMF export)
TempoMapExtractor.cs	Tempo-map extraction
TempoMap.cs / TempoMapEntry.cs / TimeSignatureEntry.cs	Data model
MeasureTimeUtility.cs	Measure / bar tick helpers
TupletUtility.cs / SwingUtility.cs	Tuplet and swing timing
MidiMetaMessageFactory.cs	SMF meta builders

Related docs

- [SMF / Sequencing \(jp.kshoji.midisystem\)](#)
- [MIDI 2.0 / UMP](#)
- [Unity Ecosystem Integration — UMP sequence](#)
- [Gameplay Components](#)
- [MIDI 1.0 Runtime \(MidiManager\)](#)
- [Samples](#)